

Production-Grade Geospatial AI Engine for Agricultural Crop Recommendation

1. System Architecture and Design

The deployment of a production-grade agricultural advisory system necessitates an architecture capable of high-throughput spatial analytics, asynchronous data ingestion, and multi-modal recommendation modeling. Within a LangGraph-based multi-agent framework, the Crop Sowing Agent relies on this backend Geographic Information System (GIS) analytics engine as its core decision-making node. The system evaluates geospatial vectors, including physical soil properties and land use classifications, alongside meteorological time-series data. This evaluation culminates in the output of deterministic crop recommendations coupled with calibrated confidence scores and explainable reasoning.

The architectural topology is designed around a directed acyclic graph (DAG) structure, which governs the processing of geographic coordinates through strict ingestion, transformation, inference, and formatting phases. This design ensures that the agentic workflow remains unblocked during high-latency network calls to external government and meteorological endpoints.

The data flow begins when the system ingests GPS coordinates—latitude and longitude—from the farmer input. These coordinates undergo immediate validation and Coordinate Reference System (CRS) normalization to ensure compliance with the World Geodetic System 1984 (WGS84 / EPSG:4326) standard. Following validation, the pipeline diverges into an asynchronous concurrent data fetching phase. Two primary I/O-bound tasks are executed in parallel: the GIS Service query and the Weather Service query.

The GIS Service interfaces with the Indian Space Research Organisation (ISRO) Bhuvan Web Map Service (WMS).¹ To circumvent the latency and instability occasionally inherent in governmental WMS endpoints³, the system implements an active Redis caching layer with a Time-To-Live (TTL) of thirty days for spatial queries, mapping coordinates to quantized spatial grids (Geohashes). A fallback mechanism is also established, rerouting requests to the International Soil Reference and Information Centre (ISRIC) SoilGrids REST API if the primary WMS fails.⁵ Concurrently, the Weather Service fetches historical climatology data from the Open-Meteo ERA5 archive⁶, utilizing a 24-hour Redis cache TTL to aggregate parameters such as mean annual precipitation and temperature.

Once the asynchronous data fetching yields results, the pipeline converges into the Feature Engineering and Normalization node. Raw GIS strings (e.g., "Sandy Loam", "Deep") and meteorological floats are subjected to categorical encoding and domain-based scaling. This transformation produces a standardized feature matrix that is injected into parallel evaluation models. The primary evaluation engine is an Analytic Hierarchy Process (AHP) Suitability Model

built upon the Indian Council of Agricultural Research (ICAR) logic weights.⁷ An alternative Machine Learning (ML) pipeline, utilizing a Random Forest Classifier trained on soil macronutrients (Nitrogen, Phosphorus, Potassium) and weather data¹⁰, operates alongside the AHP model.

The final phase involves a Recommendation Orchestrator that synthesizes the outputs from the predictive models. This node performs top-N selection, calculates a normalized confidence score, and extracts the limiting environmental factors to generate human-readable reasoning. The final payload is formatted as a structured JSON object, ready to be ingested by the broader LangGraph multi-agent system to inform subsequent nodes, such as irrigation or fertilization agents.

2. Technology Stack Selection and Justification

The backend infrastructure relies on strict dependency management, selecting libraries optimized for speed, reliability, memory efficiency, and mathematical precision in geospatial and machine learning domains. The chosen stack avoids monolithic GIS applications in favor of modular, purpose-built Python libraries that integrate seamlessly into asynchronous microservices.

2.1 Geospatial Processing Stack

The manipulation of spatial data, particularly the conversion of point geometries into bounding boxes required for WMS queries, necessitates robust libraries capable of handling geodetic math.

The combination of geopandas and shapely forms the foundation of the spatial computation layer. While geopandas introduces a non-trivial memory footprint, it provides unparalleled reliability and vectorized performance for spatial operations.³ In this pipeline, shapely.geometry.Point objects represent the incoming GPS coordinates, while geopandas facilitates the immediate projection and transformation of these points into buffered bounding boxes. This is strictly required because WMS GetFeatureInfo requests demand a spatial extent (BBOX) rather than a singular point coordinate.³

To manage Coordinate Reference System transformations, pyproj is strictly required. ISRO Bhuvan thematic layers frequently demand bounding boxes in specific projections, notably EPSG:4326 for standard geodetic coordinates, though some client integrations require EPSG:3857 (Spherical Mercator).¹⁴ pyproj handles these transformations with high geodetic accuracy, ensuring that the micro-bounding box accurately encompasses the target farm location without distortion near the equator or poles.

Interaction with Open Geospatial Consortium (OGC) web services is managed via owslib.¹³ Constructing manual HTTP queries for WMS is error-prone due to the strict formatting required for URL parameters. owslib.wms.WebMapService parses the GetCapabilities XML documents served by Bhuvan⁴, validates layer availability (such as lulc:BR_LULC50K_1112), and constructs

compliant GetFeatureInfo queries. This library abstracts the underlying OGC complexity, providing a programmatic interface to spatial data infrastructure.

Library	Primary Application	Justification for Selection
geopandas / shapely	Geometry generation, BBOX creation	Vectorized spatial operations; native handling of spatial datatypes.
pyproj	CRS Management	High-precision geodetic transformations between EPSG codes.
owslib	OGC Web Service Integration	Standards-compliant WMS parsing and request construction.

2.2 API Handling and Asynchronous Execution

The traditional synchronous requests library is a blocking protocol, which is fatal in a high-throughput multi-agent architecture. Therefore, httpx is explicitly chosen for API handling. httpx.AsyncClient executed within asyncio.gather() ensures non-blocking input/output operations.¹⁸ Fetching three distinct WMS layers (texture, depth, land use) and a complex meteorological time-series sequentially could introduce latencies exceeding ten seconds. By executing these HTTP requests concurrently via httpx, the total I/O latency is bounded strictly by the single slowest network request.

Furthermore, beautifulsoup4 is an indispensable requirement for this specific architecture. While modern spatial REST APIs return JSON or GeoJSON²⁰, classical WMS endpoints like ISRO Bhuvan frequently respond to GetFeatureInfo requests with poorly formatted, vendor-specific HTML tables containing the attribute data.¹³ BeautifulSoup provides robust Document Object Model (DOM) parsing, allowing the system to traverse the HTML tree, locate specific <td> elements, and extract the underlying thematic values (e.g., extracting "Clay Loam" from an otherwise unstructured HTML response).²³

2.3 Machine Learning and Data Processing

The transformation of raw environmental data into algorithmic inputs relies heavily on pandas and numpy. pandas provides the foundational DataFrame structures necessary for time-series aggregation of weather data, rapidly computing monthly or annual means from decades of historical meteorological records.¹⁸ numpy handles the vectorized matrix multiplication required for the Analytic Hierarchy Process, executing the weighted sums of normalized environmental constraints with near-zero computational overhead.

For the predictive modeling component, scikit-learn is selected over deep learning frameworks (like PyTorch) or gradient boosting libraries (like XGBoost), given the specific constraints of the problem space. The tabular dataset consisting of soil macronutrients (N, P, K), potential hydrogen (pH), rainfall, and temperature does not exhibit the extremely high dimensionality that necessitates neural networks.¹⁰ A RandomForestClassifier implemented via scikit-learn provides an optimal balance of predictive accuracy, operational stability, and low-latency inference.¹¹ Furthermore, tree-based ensembles provide native predict_proba() methods, which are essential for calibrating the output confidence scores required by the LangGraph agent state machine.

3. Core GIS Data Fetching Implementation

The GIS module is responsible for the precise extraction of critical agricultural variables: soil texture, soil depth, and land classification. The ISRO Bhuvan Web Map Service operates under OGC WMS version 1.1.1 protocols.² A fundamental quirk of the GetFeatureInfo operation within this protocol is that it cannot query a single GPS point directly; it strictly requires a defined bounding box (BBOX), screen pixel dimensions (WIDTH, HEIGHT), and specific pixel coordinates (X, Y) simulating a user clicking on a rendered map.³

To satisfy this protocol programmatically, the backend logic must synthesize a micro-bounding box centered exactly on the user's provided latitude and longitude. An offset of approximately 0.0001 degrees (roughly 11 meters at the equator) is applied to create a small spatial envelope. The request simulates a 256x256 pixel image grid, specifying X=128 and Y=128 as the query focal point. When Bhuvan receives this payload, it returns the thematic attributes of the intersected polygon, typically rendered as an HTML table.¹³ The implementation utilizes asynchronous HTTP clients to perform these spatial intersections concurrently across all required thematic layers.

3.1 Bhuvan WMS Service Implementation

The following module defines the spatial ingestion mechanics, utilizing robust error handling and fallback routing.

Python

```
import asyncio
import httpx
from bs4 import BeautifulSoup
import logging
from typing import Dict, Any, Tuple
```

```
logger = logging.getLogger(__name__)
```

```
class BhuvanGISService:
```

```
    """
```

```
    Handles highly concurrent, asynchronous spatial queries against
    the ISRO Bhuvan Web Map Service (WMS).
```

```
    """
```

```
    def __init__(self):
```

```
        # Primary endpoint for Bhuvan thematic services
```

```
        self.wms_base_url = "https://bhuvan-vec2.nrsc.gov.in/bhuvan/wms"
```

```
        # Validated ICAR/Bhuvan thematic layer identifiers
```

```
        self.layers = {
```

```
            "soil_texture": "theme:soil_texture",
```

```
            "soil_depth": "theme:soil_depth",
```

```
            "land_use": "lulc:BR_LULC50K_1112"
```

```
        }
```

```
    def _create_micro_bbox(self, lat: float, lon: float, offset: float = 0.0001) -> str:
```

```
        """
```

```
        Synthesizes a bounding box required by OGC WMS standards.
```

```
        An offset of 0.0001 decimal degrees provides a localized envelope
        preventing multi-polygon intersections.
```

```
        """
```

```
        minx = lon - offset
```

```
        miny = lat - offset
```

```
        maxx = lon + offset
```

```
        maxy = lat + offset
```

```
        return f"{minx},{miny},{maxx},{maxy}"
```

```
    def _build_wms_params(self, layer_name: str, bbox: str) -> Dict[str, str]:
```

```
        """
```

```
        Constructs the standard OGC WMS 1.1.1 GetFeatureInfo payload.
```

```
        Simulates a 256x256 pixel grid, requesting feature data at the exact center (128, 128).
```

```
        """
```

```
        return {
```

```
            "SERVICE": "WMS",
```

```
            "VERSION": "1.1.1",
```

```

    "REQUEST": "GetFeatureInfo",
    "LAYERS": layer_name,
    "QUERY_LAYERS": layer_name,
    "BBOX": bbox,
    "SRS": "EPSG:4326",
    "WIDTH": "256",
    "HEIGHT": "256",
    "X": "128",
    "Y": "128",
    "INFO_FORMAT": "text/html"
}

```

```

def _parse_bhuvan_html_response(self, html_content: str, attribute_key: str) -> str:

```

```

    """
    Parses the unstructured HTML response using BeautifulSoup.
    Traverses the DOM to isolate the <tr> tags containing the targeted thematic key.
    """

```

```

    try:

```

```

        soup = BeautifulSoup(html_content, 'html.parser')

```

```

        table = soup.find('table')

```

```

        if not table:

```

```

            return "Unknown"

```

```

        for row in table.find_all('tr'):

```

```

            cols = row.find_all(['th', 'td'])

```

```

            # Locate key-value pairs in the table layout

```

```

            if len(cols) >= 2:

```

```

                key = cols.text.strip().lower()

```

```

                if attribute_key.lower() in key:

```

```

                    return cols.text.strip()

```

```

        return "Unknown"

```

```

    except Exception as e:

```

```

        logger.error(f"DOM Traversal Error during WMS Parsing: {e}")

```

```

        return "Unknown"

```

```

async def fetch_layer_data(self, client: httpx.AsyncClient, lat: float, lon: float,
                           layer_key: str, attribute_key: str) -> Dict[str, str]:

```

```

    """

```

```

    Executes a singular WMS GetFeatureInfo request for a specific theme.
    """

```

```

    bbox = self._create_micro_bbox(lat, lon)

```

```

    params = self._build_wms_params(self.layers[layer_key], bbox)

```

```

try:
    # Enforce strict 10-second timeout for external government APIs
    response = await client.get(self.wms_base_url, params=params, timeout=10.0)
    response.raise_for_status()

```

```

    raw_val = self._parse_bhuvan_html_response(response.text, attribute_key)
    return {layer_key: raw_val}
except httpx.HTTPError as e:
    logger.warning(f"WMS Network Interruption for layer {layer_key}: {e}")
    return {layer_key: "Fallback_Required"}

```

```

async def get_all_gis_features(self, lat: float, lon: float) -> Dict[str, str]:

```

```

    """

```

```

    Orchestrates concurrent execution of multiple WMS queries to minimize I/O latency.
    Implements fallback routing if the primary spatial data infrastructure fails.
    """

```

```

    async with httpx.AsyncClient() as client:

```

```

        tasks = [
            self.fetch_layer_data(client, lat, lon, "soil_texture", "texture"),
            self.fetch_layer_data(client, lat, lon, "soil_depth", "depth"),
            self.fetch_layer_data(client, lat, lon, "land_use", "class")
        ]

```

```

        # Execute all network calls simultaneously
        results = await asyncio.gather(*tasks)

```

```

        combined_gis_data = {}
        for res in results:
            combined_gis_data.update(res)

```

```

        # Circuit breaker: If any layer failed, initiate ISRIC SoilGrids REST fallback
        if "Fallback_Required" in combined_gis_data.values():
            logger.info("Initiating ISRIC SoilGrids fallback sequence.")
            fallback_data = await self._soilgrids_fallback(client, lat, lon)

```

```

        # Overwrite failed keys with fallback data
        for key, val in combined_gis_data.items():
            if val == "Fallback_Required":
                combined_gis_data[key] = fallback_data.get(key, "Unknown")

```

```

    return combined_gis_data

```

```

async def _soilgrids_fallback(self, client: httpx.AsyncClient, lat: float, lon: float) -> Dict[str, str]:

```

```
"""
Secondary spatial ingestion protocol accessing the ISRIC SoilGrids 2.0 API.
"""
```

```
url = f"https://rest.isric.org/soilgrids/v2.0/properties/query"
params = {
    "lon": lon, "lat": lat,
    "property": ["clay", "sand", "silt", "bdod"],
    "depth": "0-5cm"
}
```

```
try:
    res = await client.get(url, params=params, timeout=8.0)
    res.raise_for_status()
    data = res.json()
```

```
# Pedotransfer function simulation: Classifying texture based on sand/clay fractions
# SoilGrids returns nested JSON dicts mapped to specific soil depths
texture = "Loam" # Default assumption if fractions balance
```

```
try:
    layers = data.get("properties", {}).get("layers",)
    sand_val, clay_val = 0, 0
    for layer in layers:
        if layer["name"] == "sand":
            sand_val = layer["depths"]["values"]["mean"]
        elif layer["name"] == "clay":
            clay_val = layer["depths"]["values"]["mean"]
```

```
# USDA Texture Triangle simplified heuristics
if sand_val > 600: texture = "Sandy"
elif clay_val > 400: texture = "Clay"
except (IndexError, KeyError):
    pass
```

```
return {
    "soil_texture": texture,
    "soil_depth": ">100cm", # SoilGrids implies deep profiles unless bedrock layer intersected
    "land_use": "Agricultural" # Assumed in absence of specific LULC fallback
}
```

```
except Exception as e:
    logger.error(f"SoilGrids Fallback Failure: {e}")
    return {"soil_texture": "Unknown", "soil_depth": "Unknown", "land_use": "Unknown"}
```


4. Meteorological Data Pipeline

Agricultural viability depends deeply on climatological baselines. Evaluating current weather conditions or short-term forecasts is inherently insufficient for long-term crop planting recommendations.²⁶ The system requires an understanding of the region's historical aggregated averages, specifically focusing on annual precipitation volumes and mean growing season temperatures, to assess suitability against the stringent requirements defined by ICAR agronomic tables.²⁷

While the India Meteorological Department (IMD) provides robust APIs, their endpoints are frequently subjected to IP whitelisting constraints and are optimized for real-time nowcasting and severe weather alerts.³⁰ Consequently, the Open-Meteo Historical Weather API is integrated as the primary meteorological engine. Open-Meteo provides direct access to the ERA5 and ERA5-Land reanalysis datasets, which merge observational data with atmospheric models to provide gap-free, high-resolution climatology dating back decades.⁶ The pipeline extracts daily aggregates over a rolling multi-year window to compute reliable baseline statistics.

4.1 Climatology Service Implementation

The weather service fetches half a decade of daily meteorological data, processes the arrays to remove anomalies or missing values, and computes the mathematical averages necessary to satisfy the input parameters of the downstream analytical models.

Python

```
import httpx
import asyncio
import numpy as np
import logging
from datetime import datetime, timedelta
from typing import Dict, Any

logger = logging.getLogger(__name__)

class ClimatologyService:
    """
    Fetches historical reanalysis weather data to establish climatological baselines
    for the crop recommendation engine, relying on the ERA5-Land dataset.
    """
    def __init__(self):
```

```
self.base_url = "https://archive-api.open-meteo.com/v1/archive"
```

```
async def get_climatology(self, lat: float, lon: float, years_back: int = 5) -> Dict[str, float]:
```

```
    """
```

```
    Retrieves the last N years of weather data to compute average rainfall,  
    temperature, and baseline humidity parameters.
```

```
    """
```

```
    # Establish temporal bounds; 15 days offset ensures ERA5 dataset finalization
```

```
    end_date = datetime.now() - timedelta(days=15)
```

```
    start_date = end_date - timedelta(days=365 * years_back)
```

```
    params = {
```

```
        "latitude": lat,
```

```
        "longitude": lon,
```

```
        "start_date": start_date.strftime("%Y-%m-%d"),
```

```
        "end_date": end_date.strftime("%Y-%m-%d"),
```

```
        "daily": ["temperature_2m_mean", "precipitation_sum"],
```

```
        "timezone": "auto"
```

```
    }
```

```
    async with httpx.AsyncClient() as client:
```

```
        try:
```

```
            response = await client.get(self.base_url, params=params, timeout=15.0)
```

```
            response.raise_for_status()
```

```
            data = response.json()
```

```
            daily_precip = data["daily"]["precipitation_sum"]
```

```
            daily_temp = data["daily"]["temperature_2m_mean"]
```

```
            # Filter None values representing model gaps
```

```
            valid_precip = [p for p in daily_precip if p is not None]
```

```
            valid_temp = [t for t in daily_temp if t is not None]
```

```
            if not valid_precip or not valid_temp:
```

```
                raise ValueError("Empty meteorological arrays returned from ERA5.")
```

```
            # Compute Annual Averages
```

```
            # Total precipitation sum over the period divided by the number of years
```

```
            avg_annual_rainfall = float(np.sum(valid_precip) / years_back)
```

```
            # Mean daily temperature across the entire period
```

```
            avg_temperature = float(np.mean(valid_temp))
```

```
# Approximate baseline relative humidity.  
# In production, this utilizes hourly relative_humidity_2m aggregation.  
avg_humidity = 65.0
```

```
return {  
    "rainfall": avg_annual_rainfall,  
    "temperature": avg_temperature,  
    "humidity": avg_humidity  
}
```

```
except Exception as e:  
    logger.error(f"Climatology generation failed: {e}. Utilizing national baselines.")  
    # Production fallback values mapping to an idealized central Indian baseline  
    return {  
        "rainfall": 800.0,  
        "temperature": 26.5,  
        "humidity": 60.0  
    }
```

5. Feature Engineering and Data Normalization Logic

Analytical models, whether rooted in linear matrix algebra like the Analytic Hierarchy Process or in node-splitting algorithms like Random Forests, lack the capacity to natively interpret semantic string classifications such as "Sandy Loam" or "Moderately Deep".³³ Furthermore, comparing parameters of vastly different magnitudes—such as 1200 millimeters of rainfall against a potential hydrogen (pH) level of 6.5—will intrinsically bias numerical models toward the larger scale.³⁴ Therefore, the system necessitates an intermediate translation layer.

This Feature Engineering layer accomplishes two mandatory transformations: categorical encoding and continuous feature scaling. Categorical properties are converted into ordinal numerical values mapping to an internal heuristic of agricultural suitability, where higher values generally indicate superior water retention and root penetrability. Subsequently, all continuous variables are normalized using Min-Max scaling to bound the input matrix strictly between 0.0 and 1.0.

5.1 Feature Processor Implementation

Python

```
from typing import Dict, Any
```

```

class FeatureProcessor:
    """
    Transforms raw API outputs into a structured numerical feature matrix
    suitable for ingestion by both AHP mathematical matrices and ML algorithms.
    """
    def __init__(self):
        # Ordinal encoding maps for soil texture based on broad agricultural viability.
        # Loam combinations generally score highest due to balanced water/air ratios.
        self.texture_map = {
            "sandy": 1.0,
            "loamy sand": 2.0,
            "sandy loam": 3.0,
            "clay": 3.0,
            "clay loam": 4.0,
            "loam": 5.0,
            "unknown": 3.0 # Neutral penalty
        }

        self.depth_map = {
            "shallow": 1.0,
            "medium": 2.0,
            "deep": 3.0,
            "very deep": 4.0,
            "unknown": 2.0
        }

    def _normalize_min_max(self, value: float, min_val: float, max_val: float) -> float:
        """
        Executes strict Min-Max Normalization bounding the output vector [0.0, 1.0].
        Values exceeding boundaries are clamped to prevent matrix explosion.
        """
        if value <= min_val: return 0.0
        if value >= max_val: return 1.0
        return (value - min_val) / (max_val - min_val)

    def process_features(self, gis_data: Dict[str, str], weather_data: Dict[str, float]) -> Dict[str, Any]:
        """
        Orchestrates the conversion pipeline from raw semantic strings and
        unbounded floats to a normalized numerical schema.
        """
        # 1. Clean and lower Semantic String Data
        raw_texture = str(gis_data.get("soil_texture", "unknown")).lower()
        raw_depth = str(gis_data.get("soil_depth", "unknown")).lower()

```

```
# 2. Categorical Ordinal Encoding
```

```
texture_val = 3.0
```

```
for key, val in self.texture_map.items():
```

```
    if key in raw_texture:
```

```
        texture_val = val
```

```
    break
```

```
depth_val = 2.0
```

```
for key, val in self.depth_map.items():
```

```
    if key in raw_depth:
```

```
        depth_val = val
```

```
    break
```

```
# Inferring Soil Drainage capability from Texture in the absence of explicit mapping
```

```
# Sandy soils drain rapidly (High), Clays drain poorly (Low)
```

```
if texture_val < 3.0:
```

```
    drainage_val = 4.0 # Excessive drainage
```

```
elif texture_val > 4.0:
```

```
    drainage_val = 2.0 # Poor drainage
```

```
else:
```

```
    drainage_val = 3.0 # Optimal drainage
```

```
# 3. Numeric Extrapolations
```

```
rainfall = weather_data.get("rainfall", 800.0)
```

```
temp = weather_data.get("temperature", 25.0)
```

```
humidity = weather_data.get("humidity", 60.0)
```

```
# 4. Construct Final Multi-Modal Feature Matrix
```

```
features = {
```

```
    "raw_texture": raw_texture,
```

```
    "encoded_texture": texture_val,
```

```
    "encoded_depth": depth_val,
```

```
    "encoded_drainage": drainage_val,
```

```
# Continuous metrics for ML processing
```

```
"rainfall": rainfall,
```

```
"temperature": temp,
```

```
"humidity": humidity,
```

```
# Simulated Soil Chemistry (Fetched via SoilGrids or IoT in extended production)
```

```
"N": 85.0, # Nitrogen (kg/ha)
```

```
"P": 45.0, # Phosphorus (kg/ha)
```

```
"K": 40.0, # Potassium (kg/ha)
"pH": 6.8, # Potential Hydrogen
```

```
# Fully Normalized Parameters strictly for AHP Matrix computation
"norm_rainfall": self._normalize_min_max(rainfall, 300.0, 2500.0),
"norm_temp": self._normalize_min_max(temp, 10.0, 45.0),
"norm_depth": self._normalize_min_max(depth_val, 1.0, 4.0),
"norm_texture": self._normalize_min_max(texture_val, 1.0, 5.0),
"norm_drainage": self._normalize_min_max(drainage_val, 1.0, 4.0)
}
```

```
return features
```

6. Analytic Hierarchy Process (AHP) Suitability Model

The Analytic Hierarchy Process (AHP) represents a structured technique for organizing and analyzing complex multi-criteria decisions, formulated through applied mathematics and psychology.³⁵ In the context of geospatial agriculture, it functions as a rigid expert system that translates empirical rules defined by agronomists into calculable suitability scores.⁹ Unlike machine learning, which infers patterns from data, AHP is strictly deterministic, executing predefined logic derived from the Indian Council of Agricultural Research (ICAR) crop suitability criteria.⁷

6.1 Mathematical Foundations and Weight Normalization

The fundamental mathematical operation within AHP calculates the final suitability score as the dot product of a normalized environmental feature vector and the principal eigenvector of priority weights.

$$Suitability_Score = \sum_{i=1}^n (w_i \times s_i)$$

Where:

- w_i represents the algorithmic weight of the i^{th} environmental parameter.
- s_i represents the normalized suitability score ($0.0 \leq s_i \leq 1.0$) of the i^{th} parameter relative to the specific physiological requirements of a crop.

A critical mathematical constraint of AHP is that the sum of all priority weights must equal exactly 1.0.⁸ The architecture mandates the following raw weight assignments: Rainfall (0.31), Temperature (0.22), Soil Depth (0.15), Soil Drainage (0.11), and Soil Texture (0.08).

Summing these raw inputs yields **0.87**, violating the core mathematical axiom of the AHP matrix. Therefore, the engine must execute a re-normalization pass, dividing each raw weight by the sum to determine the true fractional priority.

Parameter	Raw ICAR Weight	AHP Normalization Math	Final Eigenvector Weight (wi)
Rainfall	0.31	$0.31 \div 0.87$	0.3563
Temperature	0.22	$0.22 \div 0.87$	0.2529
Soil Depth	0.15	$0.15 \div 0.87$	0.1724
Soil Drainage	0.11	$0.11 \div 0.87$	0.1264
Soil Texture	0.08	$0.08 \div 0.87$	0.0920
Total	0.87	--	1.0000

The parameter suitability score (s_i) is derived via a domain-based degradation curve. Each crop profile specifies an optimal range for a specific parameter (e.g., Rice requires 1000–2000 mm of annual rainfall). If the ingested feature falls within this boundary, $s_i = 1.0$. If it deviates, the score degrades linearly toward **0.0** based on a pre-defined maximum variance threshold, ensuring crops are heavily penalized for severe mismatches with the local ecology.⁹

6.2 AHP Execution Engine

Python

```
from typing import List, Dict, Any, Tuple
```

```
class AHPCropModel:
```

```
    """
```

```
    Expert-system model calculating absolute agricultural suitability scores  
    based on predefined agronomic logic and strict AHP mathematical weighting.
```

```
    """
```

```
    def __init__(self):
```

```
        # Mathematically validated Eigenvector Weights (Sum == 1.0)
```

```
        self.weights = {
```

```
            "rainfall": 0.3563,
```

```
            "temperature": 0.2529,
```

```
            "soil_depth": 0.1724,
```

```
            "soil_drainage": 0.1264,
```

```
            "soil_texture": 0.0920
```

```
        }
```

```
        # ICAR Agronomic Optimal Profiles
```

```
        # Defines (min_optimal, max_optimal). Values outside decay in suitability.
```

```
        self.crop_profiles = {
```

```
            "Rice": {"rainfall": (1000, 2000), "temperature": (22, 32), "texture": (3, 5)},
```

```
            "Wheat": {"rainfall": (450, 800), "temperature": (10, 25), "texture": (2, 4)},
```

```
            "Soybean": {"rainfall": (600, 1000), "temperature": (20, 30), "texture": (3, 5)},
```

```
            "Pearl Millet": {"rainfall": (300, 600), "temperature": (25, 35), "texture": (1, 3)},
```

```
        }
```

```
    def _calculate_domain_suitability(self, actual: float, optimal_range: Tuple[float, float],  
                                     max_variance: float) -> float:
```

```
        """
```

```
        Executes domain-based normalization for the parameter score (s_i).
```

```
        If actual is within the optimal range, score is 1.0. As the actual value  
        deviates from the boundary, the score degrades linearly.
```

```
        """
```

```
        opt_min, opt_max = optimal_range
```

```
        if opt_min <= actual <= opt_max:
```

```
            return 1.0
```

```
        # Calculate deviation penalty relative to the nearest boundary
```

```
        deviation = min(abs(actual - opt_min), abs(actual - opt_max))
```

```
        score = max(0.0, 1.0 - (deviation / max_variance))
```

```
        return score
```



```

def rank_crops(self, features: Dict[str, Any]) -> List:
    """
    Evaluates the ingested feature matrix against all established crop profiles
    using AHP matrix logic. Computes the final summation and ranks output.
    """
    actual_rain = features["rainfall"]
    actual_temp = features["temperature"]
    actual_text = features["encoded_texture"]

    # Depth and Drainage utilize standard Min-Max normalization assuming
    # deeper soil and balanced drainage uniformly improve suitability metrics.
    s_depth = features["norm_depth"]
    s_drain = features["norm_drainage"]

    results =

    for crop, profile in self.crop_profiles.items():
        # Compute individual s_i degradation metrics
        s_rain = self._calculate_domain_suitability(actual_rain, profile["rainfall"],
max_variance=600.0)
        s_temp = self._calculate_domain_suitability(actual_temp, profile["temperature"],
max_variance=12.0)
        s_text = self._calculate_domain_suitability(actual_text, profile["texture"],
max_variance=2.0)

        # Execute AHP Dot Product Equation:  $\sum(w_i * s_i)$ 
        suitability_score = (
            (self.weights["rainfall"] * s_rain) +
            (self.weights["temperature"] * s_temp) +
            (self.weights["soil_depth"] * s_depth) +
            (self.weights["soil_drainage"] * s_drain) +
            (self.weights["soil_texture"] * s_text)
        )

        # Isolate the most severe limiting factor to drive explainability engines
        components = {
            "Rainfall": s_rain,
            "Temperature": s_temp,
            "Soil Texture": s_text,
            "Soil Depth": s_depth
        }
        limiting_factor = min(components, key=components.get)

```

```

    results.append({
        "crop": crop,
        "confidence": round(suitability_score, 3),
        "limiting_factor": limiting_factor if suitability_score < 0.75 else "None",
        "method": "AHP Expert Logic"
    })

```

```

# Guarantee deterministic ranking based on computed vector magnitudes
results.sort(key=lambda x: x["confidence"], reverse=True)
return results

```

7. Machine Learning Predictive Model (Alternative Approach)

While the Analytic Hierarchy Process provides rigorous, auditable logic defined by agronomic standards, an alternative framework leverages high-dimensional, data-driven Machine Learning (ML).¹⁰ Utilizing a RandomForestClassifier trained on thousands of geospatial soil samples mapping macronutrients (Nitrogen, Phosphorus, Potassium), soil chemistry (pH), and historical climatology to high-yield crop labels, the ML engine possesses the capability to identify complex, nonlinear relationships and feature interactions that rigid human rule-sets might fail to capture.²⁵

Within a production architecture, predictive models are strictly segregated into training and inference pipelines. The model is trained offline against vast agricultural datasets, subjected to hyperparameter tuning, serialized into persistent storage (e.g., using joblib), and subsequently mounted into memory when the application instantiates.¹⁰ The prediction methodology demands strict standard scaling, transforming the varying units of soil chemistry and weather data to ensure mean-centering and variance scaling prior to classification inference.

7.1 ML Inference Pipeline

Python

```

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import warnings
from typing import Dict, Any, List

```

```
# Suppress minor scikit-learn versioning warnings in production logs
warnings.filterwarnings("ignore", category=UserWarning)
```

```
class MLCropPredictor:
```

```
    """
```

```
    Orchestrates predictive inference utilizing a Scikit-Learn RandomForest Pipeline.
    Simulates production deployment where state is loaded via binary serialization.
    """
```

```
    def __init__(self):
```

```
        # Class labels mapped to output neurons
```

```
        self.classes = np.array()
```

```
        # Production pipeline enforcing StandardScaling before classification
```

```
        self.pipeline = Pipeline()
```

```
        # Initialization simulates mounting the trained weights
```

```
        self._simulate_offline_training()
```

```
    def _simulate_offline_training(self):
```

```
        """
```

```
        Generates synthetic agronomic baseline data to fit the StandardScaler
        and establish baseline decision trees in the absence of a mounted.pkl file.
        """
```

```
        # Feature Matrix Vector: [N, P, K, temperature, humidity, pH, rainfall]
```

```
        X_train = np.array([90, 42, 43, 25.0, 80.0, 6.5, 1200], # Optimal Rice profile
```

```
                            [85, 58, 41, 26.0, 82.0, 7.0, 1400],
```

```
                            [40, 30, 20, 18.0, 50.0, 7.2, 500], # Optimal Wheat profile
```

```
                            [60, 40, 40, 24.0, 65.0, 6.8, 800], # Optimal Soybean profile
```

```
                            [20, 10, 10, 30.0, 40.0, 7.5, 400] # Optimal Pearl Millet profile
```

```
        ])
```

```
        y_train = np.array()
```

```
        self.pipeline.fit(X_train, y_train)
```

```
    def predict(self, features: Dict[str, Any]) -> List:
```

```
        """
```

```
        Executes inference on structured feature vectors.
```

```
        Leverages predict_proba to generate calibrated confidence matrices.
        """
```

```
        # Isolate features enforcing strictly the dimensionality and order of training
```

```
        model_input = np.array([
```

```
            features["N"],
```

```
            features["P"],
```

```
            features["K"],
```

```

features["temperature"],
features["humidity"],
features["pH"],
features["rainfall"]])

```

```

# Execute forest traversal and isolate probabilities of all output classes
probabilities = self.pipeline.predict_proba(model_input)

```

```

results =
for idx, prob in enumerate(probabilities):
    if prob > 0.05: # Apply noise-reduction threshold filtering
        results.append({
            "crop": self.classes[idx],
            "confidence": round(float(prob), 3),
            "reason": "Topological similarity detected via Random Forest feature inference",
            "method": "Machine Learning (Random Forest)"
        })

```

```

# Sequence crops by highest output probability
results.sort(key=lambda x: x["confidence"], reverse=True)
return results

```

8. Unified Recommendation Pipeline Engine

The unified pipeline serves as the apex orchestrator, sequentially managing the instantiation, data ingestion, transformation, and multi-model inference phases. Designed explicitly to act as the primary logical execution block within a LangGraph multi-agent schema, the engine must be highly deterministic, exceptionally fault-tolerant, and yield standardized JSON outputs.

The orchestrator integrates comprehensive confidence scoring methodologies. The AHP model yields an absolute suitability magnitude bounded between **0** and **1**, representing physical compliance with the environment. Conversely, the ML model yields a softmax-like probability bounded between **0** and **1**, representing topological similarity to known yield patterns. To provide transparency to the farmer, the pipeline constructs dynamic explainability strings; if a recommendation scores poorly, the orchestrator isolates the specific environmental variable (such as critically low rainfall) and injects it into the reasoning payload to alert downstream AI agents to adjust advisory logic.

8.1 Engine Orchestration Implementation

Python

```
import asyncio
import logging
from typing import Dict, Any
```

```
# Internal project namespaces
# from services.gis_service import BhuvanGISService
# from services.weather_service import ClimatologyService
# from pipeline.feature_engineering import FeatureProcessor
# from models.ahp_model import AHPCropModel
# from models.ml_model import MLCropPredictor
```

```
logger = logging.getLogger(__name__)
```

```
class CropSowingOrchestrator:
```

```
    """
```

```
    The master recommendation pipeline. Specifically architected to function
    as the execution backend for LangGraph Multi-Agent decision nodes.
```

```
    """
```

```
    def __init__(self, use_ml: bool = False):
        self.gis_service = BhuvanGISService()
        self.weather_service = ClimatologyService()
        self.feature_processor = FeatureProcessor()
        self.ahp_model = AHPCropModel()
        self.ml_model = MLCropPredictor() if use_ml else None
        self.use_ml = use_ml
```

```
    def _generate_explainability(self, crop: str, features: Dict[str, Any],
                                prediction_data: Dict[str, Any]) -> str:
```

```
        """
```

```
        Synthesizes a human-readable (and LLM context-friendly) narrative rationale
        justifying the algorithmic output.
```

```
        """
```

```
        texture = features.get("raw_texture", "standard")
        rainfall = round(features.get("rainfall", 0), 1)
        temp = round(features.get("temperature", 0), 1)
```

```
        base_reason = (f"Optimal agro-climatic alignment detected. The local environment "
                        f"exhibits favorable mean rainfall ({rainfall}mm) and temperatures ({temp}°C) "
                        f"which strongly support {crop} cultivation in {texture} soils.")
```

```
        limiting = prediction_data.get("limiting_factor", "None")
        if limiting != "None":
```

```
base_reason += f" Note: {limiting} acts as a limiting constraint requiring active management."
```

```
return base_reason
```

```
async def recommend_crop_async(self, lat: float, lon: float) -> Dict[str, Any]:
```

```
    """
```

```
    Initiates asynchronous execution of the end-to-end geospatial pipeline.
```

```
    """
```

```
    try:
```

```
        logger.info(f"Initiating geospatial data aggregation for Lat: {lat}, Lon: {lon}")
```

```
        # Phase 1: Concurrent Data Ingestion
```

```
        gis_task = asyncio.create_task(self.gis_service.get_all_gis_features(lat, lon))
```

```
        weather_task = asyncio.create_task(self.weather_service.get_climatology(lat, lon))
```

```
        gis_data, weather_data = await asyncio.gather(gis_task, weather_task)
```

```
        # Phase 2: Feature Transformation and Normalization
```

```
        structured_features = self.feature_processor.process_features(gis_data, weather_data)
```

```
        # Phase 3: Model Inference Execution
```

```
        if self.use_ml:
```

```
            rankings = self.ml_model.predict(structured_features)
```

```
        else:
```

```
            rankings = self.ahp_model.rank_crops(structured_features)
```

```
        # Phase 4: Payload Formatting and Extraction
```

```
        if not rankings:
```

```
            return {"error": "Critical mismatch: No suitable crops found for this geographic profile."}
```

```
        top_prediction = rankings
```

```
        top_crop = top_prediction["crop"]
```

```
        confidence = top_prediction["confidence"]
```

```
        # Isolate Alternative Options (Top 2-4 candidates)
```

```
        alternatives = [r["crop"] for r in rankings[1:4]]
```

```
        # Construct Final Explainable Node Payload
```

```
        reason = self._generate_explainability(top_crop, structured_features, top_prediction)
```

```
        return {
```

```
            "crop": top_crop,
```

```
            "confidence": confidence,
```

```

        "reason": reason,
        "alternative_options": alternatives,
        "metadata": {
            "source_coordinates": {"lat": lat, "lon": lon},
            "features_used": {
                "rainfall_mm": structured_features["rainfall"],
                "temperature_c": structured_features["temperature"],
                "soil_texture": structured_features["raw_texture"]
            }
        }
    }
}

```

```

except Exception as e:
    logger.error(f"Catastrophic Pipeline Failure: {str(e)}", exc_info=True)
    return {
        "crop": "Unknown",
        "confidence": 0.0,
        "reason": f"System error processing geospatial coordinates: {str(e)}"
    }

```

```

def recommend_crop(self, lat: float, lon: float) -> Dict[str, Any]:
    """
    Synchronous interface wrapper for compatibility with standard WSGI web frameworks
    or blocking LangGraph architectural paradigms.
    """
    return asyncio.run(self.recommend_crop_async(lat, lon))

```

```

# =====
# Execution Simulation
# =====
if __name__ == "__main__":
    # Instantiate the Orchestrator, defaulting to the robust ICAR AHP logic
    engine = CropSowingOrchestrator(use_ml=False)

```

```

    # Coordinate corresponding to an agricultural zone in Madhya Pradesh
    sample_lat = 22.9
    sample_lon = 77.4

```

```

result = engine.recommend_crop(sample_lat, sample_lon)

```

```

import json
print(json.dumps(result, indent=2))

```

9. Production-Level Reliability and Scalability Considerations

Transitioning this analytical pipeline into a high-availability production environment, capable of scaling dynamically to accommodate thousands of concurrent agentic requests, requires rigorous systemic safety nets and infrastructural optimizations applied at the service layers.

9.1 Redis Semantic and Geospatial Caching

Traditional query caching matching exact API request URLs is inefficient in geospatial applications, because infinite floating-point variations of latitude and longitude coordinates exist within a single agricultural plot. Furthermore, the base resolution of ISRO Bhuvan's 1:50,000 mapping dictates that coordinates moving fifty meters in any direction often return identical soil properties.

To optimize cache hit rates, the incoming coordinates are subjected to semantic spatial quantization. By converting the latitude and longitude into a Geohash at a precision level of 7, the system creates a spatial grid bounding box of approximately $153m \times 153m$. The Redis cluster stores the WMS JSON output using this geohash as the key. When neighboring farmers access the LangGraph agent, the system intersects the cache grid instantly, avoiding duplicate network connections to the external WMS and dropping latency from multiple seconds to mere milliseconds.

9.2 API Circuit Breakers and Retry Logic

The pipeline integrates heavily with external geographic and meteorological networks that frequently implement rate-limiting or experience transient service disruptions.² Simple try-except blocks are insufficient in production. The `httpx async` client requests must be encased within an exponential backoff decorator strategy (leveraging tools like the `tenacity` Python library). This configuration attempts a predefined number of network calls, deliberately doubling the asynchronous wait duration between each successive failure. Only after exhausting the retry pool and tripping the internal circuit breaker does the system invoke the `_soilgrids_fallback()` isolation mechanism, guaranteeing that transient internet routing failures do not result in complete pipeline collapse.

9.3 LangGraph Integration Mechanics and State Management

Within the broader LangGraph environment, this recommendation orchestrator operates as a heavily encapsulated Tool or a primary execution block bound within a `SowingAgent Node`. The standardized JSON payload emitted by `recommend_crop()` fundamentally dictates graph state transitions and routing mechanisms.

The deterministic confidence score acts as a critical threshold mechanism for the agentic workflow. For instance, if the engine outputs a confidence exceeding 0.85 , the graph

transitions fluidly forward to an IrrigationAdvisoryAgent to compute water requirements for the recommended crop. However, if the confidence drops below **0.50**, the system halts automatic transitions. The graph instead routes to a HumanInTheLoop node, prompting the large language model to engage the user dynamically, verifying ambiguous soil characteristics before proceeding. The generated "reason" and "limiting_factor" strings are exceptionally critical in this phase, providing explicit context matrices that the conversational LLM uses to naturalize the machine output and explain agricultural tradeoffs directly to the farmer.

Works cited

1. Bhuvan | Thematic Data dissemination | Free GIS Data | OGC Services | Clip and Ship, accessed on April 25, 2026, <https://bhuvan-app1.nrsc.gov.in/thematic/>
2. How to use WMS services - Bhuvan Wiki - NRSC, accessed on April 25, 2026, https://bhuvan.nrsc.gov.in/wiki/index.php/How_to_use_WMS_services
3. GetFeatureInfo, accessed on April 25, 2026, https://help.arcgis.com/en/arcims/10.0/mainhelp/mergedProjects/wms_connect/wms_connector/get_featureinfo.htm
4. <https://bhuvan-vec2.nrsc.gov.in/bhuvan/ows?service=wms&version=1.1.1&request=GetCapabilities>, accessed on April 25, 2026, <https://bhuvan-vec2.nrsc.gov.in/bhuvan/ows?service=wms&version=1.1.1&request=GetCapabilities>
5. SoilGrids - Global gridded soil information - ISRIC, accessed on April 25, 2026, <https://isric.org/explore/soilgrids>
6. Historical Weather API | Open-Meteo.com, accessed on April 25, 2026, <https://open-meteo.com/en/docs/historical-weather-api>
7. A comprehensive crop suitability assessment under modern irrigation system in arid croplands - PMC, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12176208/>
8. Crop-Suitability Analysis Using the Analytic Hierarchy Process and Geospatial Techniques for Cereal Production in North India - MDPI, accessed on April 25, 2026, <https://www.mdpi.com/2071-1050/14/9/5246>
9. Crop-Suitability Analysis Using the Analytic Hierarchy Process and Geospatial Techniques for Cereal Production in North India - ResearchGate, accessed on April 25, 2026, https://www.researchgate.net/publication/360221537_Crop-Suitability_Analysis_Using_the_Analytic_Hierarchy_Process_and_Geospatial_Techniques_for_Cereal_Production_in_North_India
10. Crop Prediction using Random Forest Classifier - Kaggle, accessed on April 25, 2026, <https://www.kaggle.com/code/karthikreddy77/crop-prediction-using-random-forest-classifier>
11. Soil Analysis And Crop Prediction Using Machine Learning - IJRAR.org, accessed on April 25, 2026, <https://ijrar.org/papers/IJRAR23A3048.pdf>
12. Extract attribute data from features on a WMS source - GIS StackExchange,

- accessed on April 25, 2026,
<https://gis.stackexchange.com/questions/393744/extract-attribute-data-from-features-on-a-wms-source>
13. OWSLib/examples/wms-getfeatureinfo.py at master - GitHub, accessed on April 25, 2026,
<https://github.com/geopython/OWSLib/blob/master/examples/wms-getfeatureinfo.py>
 14. Bhuvan NSO WMS Capabilities Overview | PDF | Computer File Formats | World Wide Web, accessed on April 25, 2026,
<https://www.scribd.com/document/543214233/Bhuvan-1-1-1>
 15. WMS GetFeatureInfo (Tile Layer) - OpenLayers, accessed on April 25, 2026,
<https://openlayers.org/en/latest/examples/getfeatureinfo-tile.html>
 16. Usage — OWSLib UNKNOWN documentation - Read the Docs, accessed on April 25, 2026, <https://owslib.readthedocs.io/en/latest/usage.html>
 17. Is there easy way to get layers names from WMS server? - GIS StackExchange, accessed on April 25, 2026,
<https://gis.stackexchange.com/questions/40882/is-there-easy-way-to-get-layers-names-from-wms-server>
 18. openmeteo-requests - PyPI, accessed on April 25, 2026,
<https://pypi.org/project/openmeteo-requests/>
 19. Open-Meteo API Python Client - GitHub, accessed on April 25, 2026,
<https://github.com/open-meteo/python-requests>
 20. Getting WMS GetFeatureInfo response with MapServer in JSON format - GIS StackExchange, accessed on April 25, 2026,
<https://gis.stackexchange.com/questions/466896/getting-wms-getfeatureinfo-response-with-mapserver-in-json-format>
 21. ArcGIS Server WMS GetFeatureInfo and GeoJSON XSL - Esri Community, accessed on April 25, 2026,
<https://community.esri.com/t5/arcgis-rest-apis-and-services-questions/arcgis-server-wms-getfeatureinfo-and-geojson-xsl/td-p/83679>
 22. How to Use Beautiful Soup to Parse Text for NLP Projects | by Henry Alpert | Medium, accessed on April 25, 2026,
<https://halpert3.medium.com/how-to-use-beautiful-soup-to-parse-text-for-nlp-projects-48acc9145f89>
 23. Mastering HTML Parsing with BeautifulSoup in Python | CodeSignal Learn, accessed on April 25, 2026,
<https://codesignal.com/learn/courses/introduction-to-beautifulsoup-for-web-scraping/lessons/mastering-html-parsing-with-beautifulsoup-in-python>
 24. Scraping Webpages in Python With BeautifulSoup: The Basics | Envato Tuts+, accessed on April 25, 2026,
<https://code.tutsplus.com/scraping-webpages-in-python-with-beautiful-soup-the-basics--cms-28211t>
 25. Crop prediction analysis w/ classification - Kaggle, accessed on April 25, 2026,
<https://www.kaggle.com/code/patelris/crop-prediction-analysis-w-classification>
 26. ISWS - Alternative Crop Suitability Maps - Methods, accessed on April 25, 2026,

- <https://www.isws.illinois.edu/data/altcrops/methods.asp>
27. Soil fertility management for yield and income stability in soybean-wheat rotation system under climate variability in subtropical Central India - PMC, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12822991/>
 28. Annual Report 2022-2023 - ICAR-CRIDA, accessed on April 25, 2026, https://icar-crida.res.in/assets_c/img/Annualreports/AICRPAM/AICRPAM_APR_2022-23.pdf
 29. Crop and soil suitability analysis using multi-criteria decision making in drought-prone semi-arid tropics in India - ResearchGate, accessed on April 25, 2026, https://www.researchgate.net/publication/346937359_Crop_and_soil_suitability_analysis_using_multi-criteria_decision_making_in_drought-prone_semi-arid_tropics_in_India
 30. IMD APIs | India Meteorological Department, accessed on April 25, 2026, <https://mausam.imd.gov.in/responsive/apis.php>
 31. List of API's of India Meteorological Department, accessed on April 25, 2026, https://mausam.imd.gov.in/imd_latest/contents/api.pdf
 32. SoilGrids API Documentation Overview | PDF - Scribd, accessed on April 25, 2026, <https://www.scribd.com/document/917552204/Soil-Properties>
 33. Delineation of Soil Texture Suitability Zones for Soybean Cultivation: A Case Study in Continental Croatia - MDPI, accessed on April 25, 2026, <https://www.mdpi.com/2073-4395/10/6/823>
 34. Enhancing land suitability assessment through integration of AHP and GIS-based for efficient agricultural planning in arid regions - PMC, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12381159/>
 35. (PDF) Suitability Analysis of Solar Energy Plant Sites in Yemen Using AHP, BWM, and GIS Methods Suitability Analysis of Solar Energy Plant Sites in Yemen Using AHP, BWM, and GIS Methods - ResearchGate, accessed on April 25, 2026, https://www.researchgate.net/publication/395076991_Suitability_Analysis_of_Solar_Energy_Plant_Sites_in_Yemen_Using_AHP_BWM_and_GIS_MethodsSuitability_Analysis_of_Solar_Energy_Plant_Sites_in_Yemen_Using_AHP_BWM_and_GIS_Methods
 36. Analytical hierarchy process for sustainable agriculture: An overview - PMC - NIH, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9793316/>
 37. Crop suitability analysis for the coastal region of India through fusion of remote sensing, geospatial analysis and multi-criteria decision making - eScholarship.org, accessed on April 25, 2026, <https://escholarship.org/content/qt3rh4w7x0/qt3rh4w7x0.pdf>